



YELIA4AP

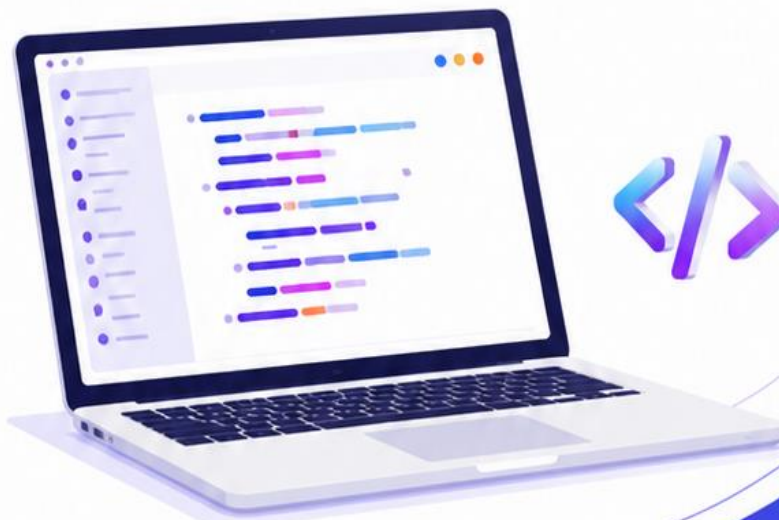
— RECURSO EDUCATIVO —

• UNIDAD 1 •

FUNDAMENTOS DE PROGRAMACIÓN ORIENTADA A OBJETOS



Asignatura:
Programación Avanzada



Índice

1.1 Introducción a la Programación Orientada a Objetos.....	3
1.2 Clases y Objetos	5
1.3 Encapsulamiento.....	8
1.4 Repaso de Comandos DDL de Base de Datos SQL	12

1.1 Introducción a la Programación Orientada a Objetos

La Programación Orientada a Objetos (POO) es uno de los enfoques más utilizados en el desarrollo de software moderno. Actualmente, muchas aplicaciones que utilizamos diariamente, como sistemas académicos, aplicaciones móviles, plataformas web y videojuegos, han sido desarrolladas utilizando este paradigma de programación.

La POO permite organizar y estructurar mejor los programas mediante la representación de elementos del mundo real en forma de objetos. Gracias a este enfoque, los sistemas pueden ser más fáciles de comprender, mantener y ampliar con el paso del tiempo. A diferencia de otros paradigmas de programación, la Programación Orientada a Objetos se enfoca en la creación de entidades que poseen características y comportamientos propios. Esto facilita el desarrollo de aplicaciones más organizadas y cercanas a la realidad.

En esta unidad se estudiarán los fundamentos de la Programación Orientada a Objetos, los conceptos de clases y objetos, el encapsulamiento y un repaso de los comandos DDL utilizados en bases de datos SQL. Estos conocimientos constituyen la base para comprender temas más avanzados que serán abordados en las siguientes unidades.

¿Qué es la Programación Orientada a Objetos?

La Programación Orientada a Objetos es un paradigma de programación que organiza el software mediante objetos. Un objeto puede representar cualquier elemento que exista en el mundo real o dentro de un sistema informático.

Cada objeto posee características propias llamadas atributos y acciones que puede realizar conocidas como métodos. Gracias a esta estructura, los programas pueden diseñarse de forma más organizada y comprensible.

La Programación Orientada a Objetos surgió como una alternativa para enfrentar la creciente complejidad de los sistemas informáticos. A medida que los programas se hicieron más grandes, fue necesario contar con mecanismos que facilitaran su organización y mantenimiento.

Actualmente, la mayoría de los lenguajes modernos, como Java, C#, Python y C++, incorporan características orientadas a objetos.

Objetivos de la Programación Orientada a Objetos

La Programación Orientada a Objetos busca alcanzar varios objetivos fundamentales:

Organizar mejor el código.

- Facilitar el mantenimiento de los programas.
- Favorecer la reutilización de componentes.
- Reducir errores durante el desarrollo.
- Permitir la creación de sistemas escalables.
- Representar situaciones reales mediante objetos.

Estos objetivos contribuyen a que el desarrollo de software sea más eficiente y adaptable a las necesidades de los usuarios.

Ventajas de la Programación Orientada a Objetos

La adopción de la POO ofrece múltiples beneficios dentro del desarrollo de software.

Mejor organización

- Los programas pueden dividirse en componentes más pequeños y fáciles de administrar.

Reutilización del código

- Los elementos desarrollados pueden utilizarse nuevamente en diferentes partes de una aplicación.

Facilidad de mantenimiento

- Las modificaciones pueden realizarse de forma localizada sin afectar todo el sistema.

Escalabilidad

- Es posible agregar nuevas funcionalidades sin necesidad de reconstruir completamente el software.

Mayor claridad

- La estructura basada en objetos facilita la comprensión del funcionamiento del sistema.
- Comparación con otros paradigmas de programación

Antes de la aparición de la Programación Orientada a Objetos, era común utilizar la programación procedimental.

La programación procedimental organiza el software mediante procedimientos o funciones que ejecutan tareas específicas. Aunque este enfoque resulta adecuado para programas pequeños, puede volverse difícil de administrar cuando el sistema crece.

La Programación Orientada a Objetos propone una organización basada en objetos que agrupan tanto los datos como las operaciones relacionadas con dichos datos.

Mientras que la programación procedimental se centra principalmente en las acciones que realiza el sistema, la POO se enfoca en las entidades que participan dentro de él. Por esta razón, la Programación Orientada a Objetos se ha convertido en uno de los paradigmas más utilizados en la actualidad.

Aplicaciones de la POO en la vida real

La Programación Orientada a Objetos se utiliza en una gran variedad de sistemas informáticos:

- Sistemas académicos universitarios.
- Aplicaciones bancarias.
- Sistemas de gestión empresarial.
- Plataformas de comercio electrónico.
- Redes sociales.
- Aplicaciones móviles.
- Videojuegos.
- Sistemas de inteligencia artificial.

Gracias a su flexibilidad y capacidad de organización, este paradigma continúa siendo una de las bases fundamentales del desarrollo de software moderno.

Idea clave de la sección

La Programación Orientada a Objetos es una forma de desarrollar software mediante objetos que representan elementos del mundo real, permitiendo crear programas más organizados, reutilizables y fáciles de mantener.

1.2 Clases y Objetos

Uno de los conceptos más importantes dentro de la Programación Orientada a Objetos es la relación entre clases y objetos. Estos elementos constituyen la base sobre la cual se construyen las aplicaciones orientadas a objetos.

Comprender estos conceptos permite representar personas, productos, estudiantes, vehículos, cuentas bancarias y cualquier otro elemento que forme parte de un sistema informático.

Las clases y los objetos trabajan de manera conjunta. Una clase define cómo será una entidad, mientras que un objeto representa una instancia concreta de dicha definición.

¿Qué es una clase?

Una clase puede entenderse como una plantilla o modelo utilizado para crear objetos. Dentro de una clase se especifican las características y comportamientos que tendrán los objetos que se generen a partir de ella.

Las clases permiten organizar la información relacionada con una entidad específica. Gracias a esto, el software puede estructurarse de forma más clara y ordenada.

Por ejemplo, en un sistema académico podría existir una clase denominada Estudiante. Esta clase contendría toda la información y funcionalidades relacionadas con los estudiantes registrados dentro del sistema.

Las clases no representan elementos reales por sí mismas. Funcionan como una definición que servirá posteriormente para crear objetos concretos.

Características de una clase

Las clases suelen contener dos componentes fundamentales:

Atributos

Representan las características o datos que posee una entidad.

**Los atributos permiten almacenar información relevante sobre un objeto.
Por ejemplo, un estudiante puede tener:**

- Nombre
- Apellido
- Edad
- Correo electrónico
- Carrera

Estos datos forman parte de los atributos de la clase.

Métodos

- Representan las acciones o comportamientos que puede realizar una entidad.
- Los métodos permiten definir funcionalidades dentro de una clase.
- Por ejemplo, un estudiante podría:
 - Registrarse
 - Consultar calificaciones
 - Actualizar información personal
 - Visualizar recursos académicos

Estas acciones constituyen ejemplos de métodos.

¿Qué es un objeto?

Un objeto es una instancia creada a partir de una clase.

Mientras la clase funciona como una plantilla, el objeto representa un elemento concreto que existe dentro del sistema.

Los objetos contienen valores específicos para cada uno de sus atributos y pueden ejecutar los métodos definidos por su clase.

En otras palabras, un objeto es la representación práctica de una clase dentro de una aplicación.

Por ejemplo, si existe una clase llamada Estudiante, se pueden crear varios objetos diferentes:

Estudiante 1
Estudiante 2
Estudiante 3

Todos pertenecen a la misma clase, pero poseen información distinta.

Relación entre clases y objetos

Las clases y los objetos mantienen una relación directa.

La clase define:

- Qué información tendrá el objeto.
- Qué acciones podrá realizar.
- Cómo estará organizada la entidad.

El objeto representa:

- Una instancia real de esa definición.
- Datos específicos.
- Comportamientos concretos.

Puede decirse que la clase actúa como un plano de construcción, mientras que el objeto corresponde al resultado obtenido a partir de dicho plano.
Sin clases no podrían existir objetos y sin objetos las clases no tendrían utilidad práctica dentro de un sistema.

Creación de objetos

Una de las principales ventajas de la Programación Orientada a Objetos es la posibilidad de crear múltiples objetos utilizando una misma clase.
Esto permite reutilizar estructuras ya definidas sin necesidad de repetir código constantemente.

Cada objeto mantiene sus propios valores y puede interactuar de manera independiente dentro del sistema.
Esta capacidad de reutilización contribuye a desarrollar aplicaciones más organizadas y fáciles de mantener.

Instanciación

El proceso mediante el cual se crea un objeto a partir de una clase recibe el nombre de instanciación.

Cuando se instancia una clase, el sistema reserva espacio en memoria para almacenar la información correspondiente al nuevo objeto.

A partir de ese momento, el objeto puede utilizar sus atributos y ejecutar los métodos definidos por la clase.

La instanciación es una de las operaciones fundamentales dentro de la Programación Orientada a Objetos, ya que permite transformar una definición abstracta en un elemento funcional dentro del programa.

Importancia de las clases y objetos

Las clases y los objetos constituyen el núcleo de la Programación Orientada a Objetos.

Gracias a ellos es posible:

- Modelar situaciones del mundo real.
- Organizar mejor la información.
- Reutilizar componentes.
- Facilitar el mantenimiento del software.
- Reducir la complejidad de los sistemas.
- Mejorar la comprensión del código.

Por estas razones, todo desarrollador que trabaje con Programación Orientada a Objetos debe comprender claramente la diferencia y relación existente entre ambos conceptos.

Idea clave de la sección

Las clases funcionan como plantillas que definen características y comportamientos, mientras que los objetos son instancias concretas creadas a partir de dichas plantillas. La relación entre ambos constituye la base fundamental de la Programación Orientada a Objetos.

1.3 Encapsulamiento

A medida que los sistemas informáticos crecen, se vuelve necesario proteger la información que manejan. No todos los datos deben estar disponibles para cualquier parte del programa, ya que esto podría provocar errores, inconsistencias o modificaciones no deseadas.

La Programación Orientada a Objetos ofrece diferentes mecanismos para controlar el acceso a la información. Uno de los más importantes es el encapsulamiento. El encapsulamiento permite proteger los datos de un objeto y controlar la manera en que pueden ser consultados o modificados. Gracias a este principio, los programas se vuelven más seguros, organizados y fáciles de mantener.

¿Qué es el encapsulamiento?

El encapsulamiento es un principio de la Programación Orientada a Objetos que consiste en proteger los datos internos de una clase, permitiendo que el acceso a ellos se realice de forma controlada.

Este principio busca evitar que cualquier parte del programa modifique la información directamente sin seguir ciertas reglas establecidas.

En otras palabras, el encapsulamiento actúa como una barrera de protección alrededor de los datos de un objeto.

Gracias a esta protección, los desarrolladores pueden controlar cómo se utilizan los atributos y garantizar que la información permanezca consistente durante la ejecución del sistema.

¿Por qué es importante el encapsulamiento?

En aplicaciones pequeñas puede parecer innecesario proteger la información. Sin embargo, cuando los sistemas crecen y participan varios desarrolladores, el riesgo de modificar datos incorrectamente aumenta considerablemente.

El encapsulamiento ayuda a evitar estos problemas al limitar el acceso directo a los atributos de una clase.

Además, permite establecer reglas para validar la información antes de almacenarla, mejorando la calidad y confiabilidad del software.

Beneficios del encapsulamiento

El uso adecuado del encapsulamiento aporta múltiples ventajas durante el desarrollo de software.

Mayor seguridad

Los datos sensibles permanecen protegidos frente a modificaciones no autorizadas.

Mejor organización

La información se administra de manera más ordenada dentro de cada clase.

Facilidad de mantenimiento

Los cambios pueden realizarse sin afectar otras partes del sistema.

Reducción de errores

Al controlar el acceso a los datos se disminuye la posibilidad de introducir información incorrecta.

Mayor reutilización

Las clases encapsuladas pueden utilizarse en distintos proyectos con menor riesgo de problemas.

Modificadores de acceso

Para aplicar el encapsulamiento se utilizan mecanismos llamados modificadores de acceso.

Los modificadores de acceso determinan quién puede visualizar o utilizar determinados atributos y métodos dentro de una clase.

Los más utilizados son:

Public

El modificador public permite que un atributo o método sea accesible desde cualquier parte del programa.

Cuando un elemento es público, puede ser utilizado libremente por otros componentes del sistema.

Generalmente se utiliza para funcionalidades que deben estar disponibles para diferentes módulos de una aplicación.

Private

El modificador private restringe completamente el acceso directo a un atributo o método desde el exterior de la clase.

Esto significa que únicamente la propia clase puede utilizar dichos elementos. Es uno de los mecanismos más utilizados para proteger información importante.

Gracias al modificador private se evita que otros componentes alteren los datos de forma inadecuada.

Protected

El modificador protected ofrece un nivel de acceso intermedio.

Los elementos protegidos pueden ser utilizados dentro de la propia clase y también por clases relacionadas mediante herencia.

Este modificador resulta útil cuando se desea compartir información con clases derivadas sin exponerla completamente al resto del sistema.

Métodos Getter y Setter

Cuando un atributo se encuentra protegido mediante encapsulamiento, normalmente se utilizan métodos especiales para acceder a él.

Estos métodos reciben los nombres de Getter y Setter.

Getter

Un Getter es un método que permite consultar o leer el valor de un atributo. Su función principal es proporcionar acceso controlado a la información almacenada dentro de una clase.

Gracias a los Getters, los datos pueden visualizarse sin exponer directamente los atributos internos.

Setter

Un Setter es un método que permite modificar el valor de un atributo. Antes de realizar la modificación, el método puede verificar que la información sea válida.

De esta manera se evita almacenar datos incorrectos que puedan afectar el funcionamiento del sistema.

Los Setters constituyen una herramienta fundamental para mantener la integridad de la información.

Encapsulamiento en aplicaciones reales

El encapsulamiento se utiliza constantemente en aplicaciones modernas.

Algunos ejemplos incluyen:

- Sistemas bancarios.
- Plataformas académicas.
- Sistemas hospitalarios.
- Aplicaciones empresariales.
- Sistemas de gestión de inventario.
- Aplicaciones móviles.

En todos estos casos es importante proteger la información para garantizar la seguridad y confiabilidad del sistema.

Por esta razón, el encapsulamiento es considerado uno de los principios fundamentales de la Programación Orientada a Objetos.

Relación del encapsulamiento con la calidad del software

Un software bien diseñado no solo debe funcionar correctamente, sino también ser fácil de mantener y ampliar.

El encapsulamiento contribuye directamente a este objetivo al permitir que cada clase administre su propia información de forma controlada.

Cuando los datos están protegidos y organizados adecuadamente, el sistema resulta más estable y menos propenso a errores.

Por ello, este principio continúa siendo una de las prácticas más recomendadas dentro del desarrollo de software moderno.

Idea clave de la sección

El encapsulamiento permite proteger la información de una clase mediante mecanismos de acceso controlado. Gracias a este principio, los programas son más seguros, organizados y fáciles de mantener.

1.4 Repaso de Comandos DDL de Base de Datos SQL

Las bases de datos son herramientas fundamentales para almacenar y organizar información dentro de los sistemas informáticos. Actualmente, aplicaciones como redes sociales, plataformas educativas, sistemas bancarios y tiendas virtuales utilizan bases de datos para gestionar grandes cantidades de información.

Para administrar una base de datos es necesario utilizar un conjunto de instrucciones conocidas como SQL (Structured Query Language), que permiten crear, modificar y eliminar estructuras de almacenamiento.

Dentro de SQL existe una categoría denominada DDL (Data Definition Language), cuya función principal es definir la estructura de la base de datos.

Los comandos DDL permiten crear tablas, modificar sus características y eliminar elementos que ya no son necesarios.

¿Qué es DDL?

DDL significa Data Definition Language o Lenguaje de Definición de Datos. Este conjunto de comandos se utiliza para definir la estructura de una base de datos.

A diferencia de otros comandos SQL que manipulan registros o consultan información, los comandos DDL trabajan sobre la estructura que almacena los datos.

Gracias al DDL es posible crear tablas, modificarlas o eliminarlas según las necesidades del sistema.

Los comandos DDL más utilizados son:

- CREATE
- ALTER
- DROP
- TRUNCATE

Comando CREATE

Se utiliza para crear nuevos elementos dentro de una base de datos.

Entre los elementos que pueden crearse se encuentran:

- Bases de datos.
- Tablas.

- Vistas.
- Índices.

Cuando una aplicación necesita almacenar información por primera vez, generalmente se utilizan instrucciones CREATE para construir la estructura inicial de almacenamiento.

Este comando representa el punto de partida en la construcción de una base de datos.

Importancia de CREATE

- Permite iniciar el diseño de una base de datos.
- Facilita la organización de la información.
- Define la estructura que utilizará el sistema.

Comando ALTER

Permite modificar estructuras existentes dentro de una base de datos.

A medida que una aplicación evoluciona, pueden surgir nuevas necesidades de almacenamiento. En estos casos resulta necesario realizar cambios sobre tablas ya existentes.

El comando ALTER permite:

- Agregar nuevos campos.
- Modificar características de campos existentes.
- Eliminar elementos específicos.
- Actualizar la estructura de una tabla.

Importancia de ALTER

- Facilita la evolución de los sistemas.
- Permite adaptar la base de datos a nuevos requerimientos.
- Evita reconstruir completamente las tablas existentes.

Comando DROP

Se utiliza para eliminar elementos completos de una base de datos.

Cuando un objeto deja de ser necesario, puede eliminarse utilizando esta instrucción.

DROP puede aplicarse sobre:

- Bases de datos.
- Tablas.
- Vistas.
- Índices.

Es importante utilizar este comando con precaución, ya que la eliminación suele ser permanente.

Importancia de DROP

- Permite eliminar estructuras innecesarias.
- Ayuda a mantener organizada la base de datos.
- Facilita la administración de recursos.

Comando TRUNCATE

Este comando elimina todos los registros almacenados dentro de una tabla, pero conserva la estructura de la misma.

Esto significa que la tabla continúa existiendo y puede seguir utilizándose posteriormente.

TRUNCATE suele emplearse cuando se desea limpiar completamente la información almacenada sin eliminar la tabla.

Importancia de TRUNCATE

- Permite vaciar tablas rápidamente.
- Conserva la estructura original.
- Facilita procesos de pruebas y mantenimiento.

Importancia del DDL en el desarrollo de software

Los comandos DDL desempeñan un papel fundamental dentro del desarrollo de aplicaciones.

Gracias a ellos es posible construir y mantener la estructura de almacenamiento utilizada por los sistemas modernos.

Sin una correcta definición de la base de datos, la información no podría organizarse de manera eficiente.

Por esta razón, todo desarrollador debe comprender los conceptos básicos relacionados con CREATE, ALTER, DROP y TRUNCATE.

Aunque la Programación Orientada a Objetos y las bases de datos son áreas diferentes, ambas trabajan conjuntamente en la mayoría de las aplicaciones actuales.

Resumen General de la Unidad

En esta unidad se estudiaron los fundamentos de la Programación Orientada a Objetos, uno de los paradigmas más utilizados en el desarrollo de software moderno.

Se analizó el concepto de POO, sus objetivos, ventajas y diferencias con otros paradigmas de programación.

Posteriormente se abordaron los conceptos de clases y objetos, comprendiendo cómo estos permiten representar entidades del mundo real dentro de un sistema informático. También se estudió el encapsulamiento, principio fundamental que ayuda a proteger la información y controlar el acceso a los datos mediante modificadores de acceso y métodos especiales.

Finalmente, se realizó un repaso de los comandos DDL de SQL, comprendiendo la función de CREATE, ALTER, DROP y TRUNCATE dentro de la administración de bases de datos.

Los conocimientos adquiridos en esta unidad constituyen la base necesaria para comprender temas más avanzados relacionados con herencia, polimorfismo, UML, MVC y acceso a bases de datos que serán desarrollados en las siguientes unidades.

Conceptos Clave

- Programación Orientada a Objetos (POO)
- Paradigma de programación
- Clase
- Objeto
- Atributo
- Método
- Instanciación
- Encapsulamiento
- Modificador public
- Modificador private
- Modificador protected
- Getter
- Setter
- SQL
- DDL
- CREATE
- ALTER
- DROP
- TRUNCATE

Bibliografía

Deitel, P., & Deitel, H. Java: How to Program.
Schildt, H. Java: The Complete Reference.
Pressman, R. Ingeniería de Software: Un Enfoque Práctico.
Elmasri, R., & Navathe, S. Fundamentals of Database Systems.
Silberschatz, A., Korth, H., & Sudarshan, S. Database System Concepts.
Oracle Academy. Conceptos de Programación Orientada a Objetos.
Oracle Database Documentation.



GRACIAS

POR TU ATENCIÓN

